LangChain

vs

LlamaIndex

Haystack

Architectures, pipelines, code and a practical comparison of the three most popular LLM application frameworks.

LangChain
Agents & orchestration

LlamaIndex
Data & RAG specialist

Haystack
Production search pipelines

A visual guide • Python edition • 2026

The big picture

Three frameworks, one goal: connect large language models to your data, tools and users.

A large language model on its own only turns text into text. Real applications need more around it: reading your documents, finding the right passages, building prompts, calling tools, remembering a conversation and running reliably in production. LangChain, LlamaIndex and Haystack all provide these building blocks, but each grew from a different starting point, and that origin still shapes how they feel to use.

	LangChain	LlamaIndex	Haystack
Started	Oct 2022, Harrison Chase	Nov 2022 as GPT Index, Jerry Liu	2019/2020 by deepset (Berlin); 2.0 in 2024
Grew out of	Chaining prompts and tools together	Indexing private data for LLMs	Neural search and question answering
Think of it as	Swiss Army knife + agent engine	The RAG specialist	Industrial search pipeline kit
Core idea	Runnables piped with LCEL; agents on LangGraph	Documents to Nodes to Indexes to Query Engines	Typed Components wired into a Pipeline graph

The common pattern: Retrieval-Augmented Generation (RAG)

Almost every app built with these frameworks follows the same two-phase RAG pattern. You index your documents once, then at question time you fetch only the most relevant chunks and hand them to the LLM. The model answers from your data instead of guessing, and you only pay for a few chunks of context.

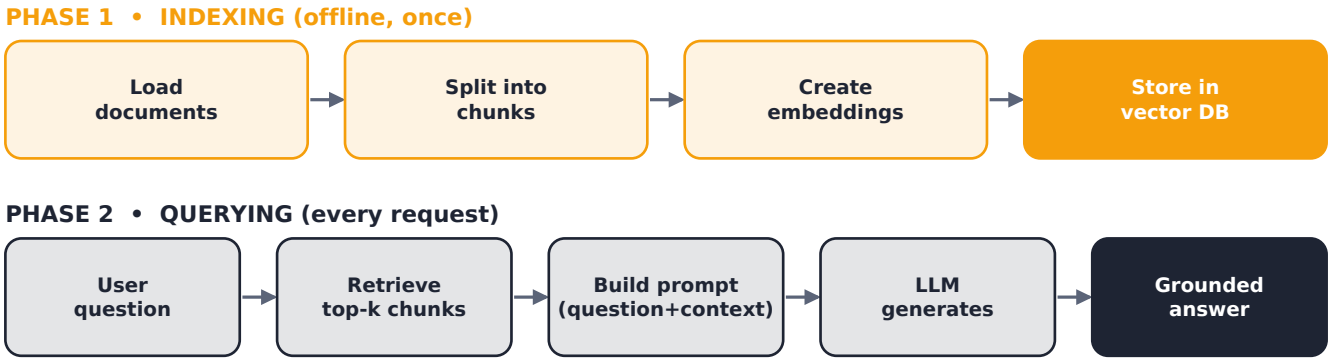


Figure 1. The generic RAG pipeline that all three frameworks implement.

Key vocabulary

Embedding: a list of numbers capturing a text's meaning, so similar texts sit close together. **Vector store**: a database that searches by meaning (FAISS, Chroma, Pinecone, Qdrant). **Chunk / Node**: a small piece of a document. **Agent**: an LLM that decides which tools to call, in a loop, until the task is done.

LangChain

Python & JavaScript
pip install langchain
Companions: LangGraph, LangSmith

The general-purpose framework for composing LLM calls, tools and agents, with the largest integration ecosystem.

LangChain gives you standard interfaces for every part of an LLM app (models, prompts, retrievers, tools) so they can be swapped and snapped together. Since version 1.0 the project has three clear layers: **langchain-core** holds the base abstractions, **langchain** provides the high-level agent and chain APIs, and dozens of **partner packages** connect to specific providers. Agents run on **LangGraph**, and **LangSmith** lets you trace and evaluate everything.

Architecture

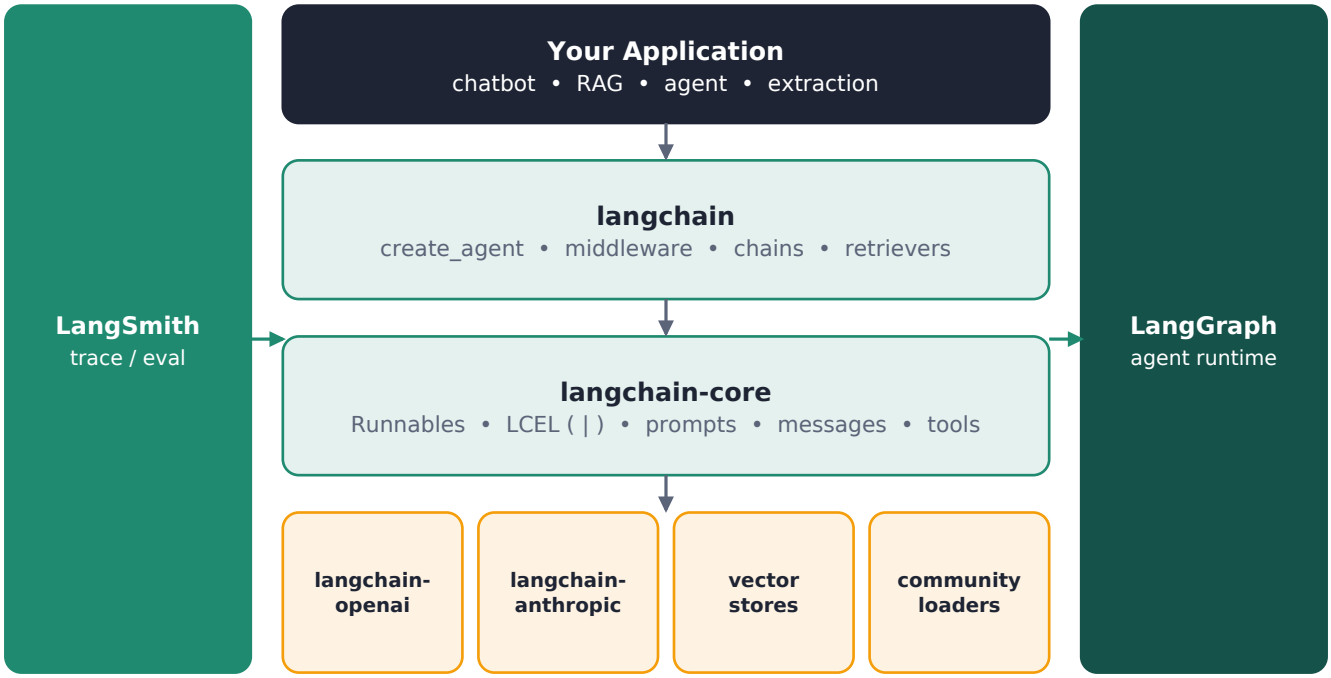


Figure 2. LangChain package layers, with LangGraph as the agent runtime and LangSmith for observability.

Key components

Chat Models

One interface for OpenAI, Anthropic, Google, Mistral, Ollama and more. Switch providers by changing one line.

Prompt Templates

Reusable prompts with {placeholders}; chat templates with system, human and AI messages.

LCEL (| operator)

LangChain Expression Language pipes Runnables: prompt | model | parser. Streaming, batching and async come free.

Output Parsers

Turn model output into strings, JSON or typed objects; structured output via schemas.

Loaders, Splitters, Retrievers

Hundreds of loaders (PDF, web, Notion), text splitters and retriever wrappers over vector stores.

Tools & Agents

Decorate a Python function with @tool; create_agent runs the reason-act loop on LangGraph.

Memory & State

Conversation history and long-running state are handled through LangGraph checkpointers.

Middleware

Hooks around the agent loop for summarisation, human approval, guardrails and retries.

How a LangChain agent works



Figure 3. The agent loop: the model repeats reason, act, observe until it can answer.

Code: a RAG chain with LCEL

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

docs   = PyPDFLoader("notes.pdf").load()
chunks = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100).split_documents(docs)
retriever = FAISS.from_documents(chunks, OpenAIEmbeddings()).as_retriever(search_kwargs={"k": 4})

prompt = ChatPromptTemplate.from_template(
    "Answer only from the context.\n\nContext:\n{context}\n\nQuestion: {question}")

rag = ({"context": retriever, "question": RunnablePassthrough()}
       | prompt | ChatOpenAI(model="gpt-4o-mini") | StrOutputParser())

print(rag.invoke("Summarise chapter 2"))
```

Code: a tool-using agent

```
from langchain.agents import create_agent
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Return the current weather for a city."""
    return f"It is 29 C and sunny in {city}."

agent = create_agent(model="openai:gpt-4o-mini", tools=[get_weather],
                     system_prompt="You are a helpful assistant.")
result = agent.invoke({"messages": [{"role": "user", "content": "Weather in Chennai?"}]})
print(result["messages"][-1].content)
```

Strengths

- Biggest ecosystem: 700+ integrations
- Best agent tooling via LangGraph
- LangSmith tracing is excellent for debugging
- Most job postings and tutorials

Watch out for

- Many layers of abstraction
- Fast-moving API: old tutorials break
- Overkill for a simple RAG app

LlamaIndex

Python & TypeScript
pip install llama-index
Companions: LlamaParse, LlamaCloud

The data framework for LLMs: best-in-class tools to ingest, index and query your own documents.

LlamaIndex was built around one question: how do you give an LLM access to data it was never trained on? It turns raw files into **Documents**, splits them into **Nodes**, organises Nodes into **Indexes**, and exposes those through **Query Engines** and **Chat Engines**. It has the richest set of retrieval strategies of the three, and newer releases add event-driven **Workflows** and agents for multi-step apps.

Architecture

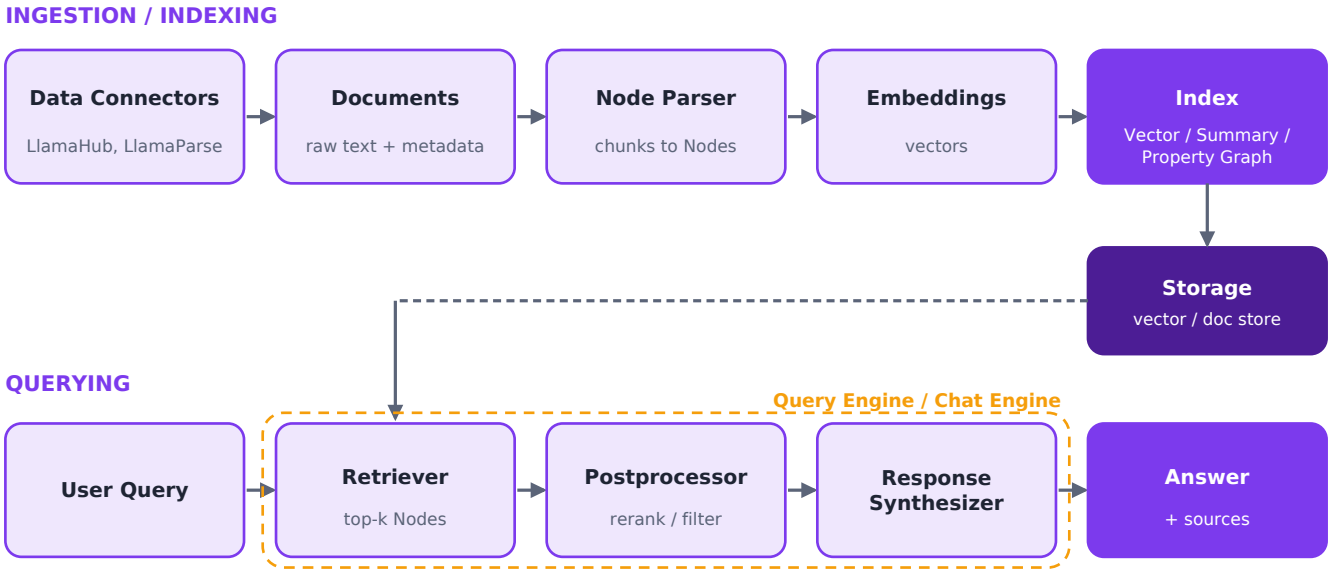


Figure 4. LlamaIndex ingestion and query paths. The Query Engine wraps retrieval, post-processing and synthesis.

Key components

Data Connectors (LlamaHub)

Hundreds of readers: PDFs, Word, Slack, Notion, SQL, APIs. SimpleDirectoryReader loads a whole folder.

LlamaParse

A hosted parser that handles complex PDFs with tables, charts and multi-column layouts.

Documents & Nodes

A Document is a whole file; Nodes are chunks that keep metadata and links to neighbours.

Indexes

VectorStoreIndex (most common), SummaryIndex, PropertyGraphIndex (knowledge graphs), keyword indexes.

Retrievers & Postprocessors

Top-k, hybrid, auto-merging, recursive retrieval; rerankers and filters clean results.

Response Synthesizers

Modes like compact, refine and tree_summarize to combine many chunks into one answer.

Query & Chat Engines

Ready-made Q&A and conversational interfaces over any index, with source citations.

Workflows & Agents

Event-driven steps for multi-step pipelines, plus agents that use query engines as tools.

Code: chat with your documents in five lines

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.llms.openai import OpenAI

Settings.llm = OpenAI(model="gpt-4o-mini")

docs  = SimpleDirectoryReader("data").load_data()        # load every file in ./data
index = VectorStoreIndex.from_documents(docs)            # chunk + embed + store
engine = index.as_query_engine(similarity_top_k=4)

response = engine.query("What are the key findings in chapter 2?")
print(response)
for src in response.source_nodes:                        # see which chunks were used
    print(src.score, src.node.metadata.get("file_name"))
```

Code: persist the index and add conversation

```
from llama_index.core import StorageContext, load_index_from_storage

index.storage_context.persist("./storage")              # save once
index = load_index_from_storage(StorageContext.from_defaults(persist_dir="./storage"))

chat = index.as_chat_engine(chat_mode="condense_plus_context")
print(chat.chat("Who wrote the report?"))
print(chat.chat("What did they recommend?"))           # remembers context
```

Strengths

- Fastest path from files to a working RAG app
- Advanced retrieval and indexing strategies
- Great document parsing with LlamaParse
- Source citations built in

Watch out for

- Less natural for non-RAG apps
- Many index types to understand
- Best parsing features are paid cloud services

Haystack

Python
pip install haystack-ai
Companions: Hayhooks, deepset AI Platform

An open-source framework by deepset for building production-ready search, RAG and agent pipelines.

Haystack treats every step as a **Component** with typed inputs and outputs, and every application as a **Pipeline**: a directed graph that connects those components explicitly. Because the wiring is explicit, pipelines are easy to read, test, validate and serialise to YAML for deployment. Version 2.0 (2024) rebuilt the framework around this design and added support for branching, loops and agents.

Architecture

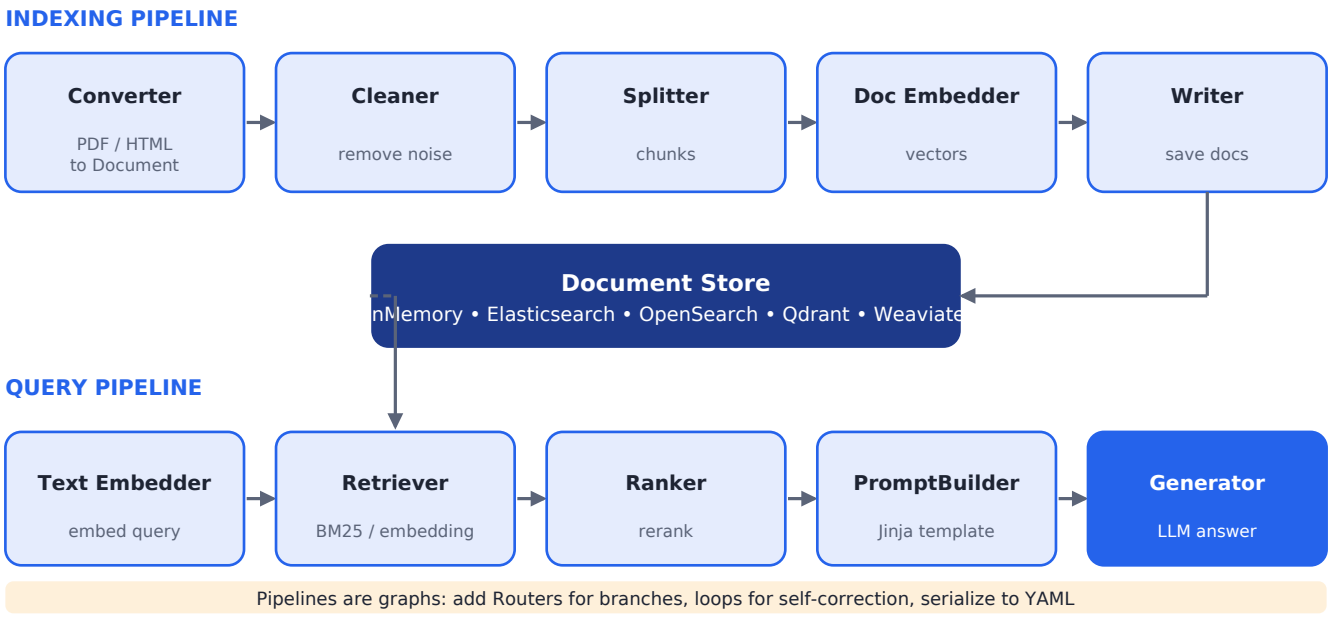


Figure 5. Haystack's two pipelines share a Document Store. Each box is a Component with typed sockets.

Key components

Components

Any Python class with @component and a run() method. Typed inputs and outputs are checked when you connect them.

Pipelines

Directed graphs with add_component() and connect(). Supports branches, loops and YAML serialisation.

Document Stores

InMemory, Elasticsearch, OpenSearch, Qdrant, Weaviate, Pinecone, pgvector and more.

Converters & Preprocessors

PDF/HTML/DOCX converters, DocumentCleaner and DocumentSplitter prepare data.

Embedders & Retrievers

Text and document embedders; BM25, embedding and hybrid retrievers.

Rankers

Cross-encoder and LLM rankers reorder retrieved documents for better precision.

Builders & Generators

PromptBuilder uses Jinja2 templates; Generators call OpenAI, Anthropic, Hugging Face, Ollama.

Routers & Agents

Routers send data down different branches; the Agent component runs tool-calling loops.

Code: a RAG query pipeline

```
from haystack import Pipeline, Document
from haystack.document_stores.in_memory import InMemoryDocumentStore
from haystack.components.retrievers.in_memory import InMemoryBM25Retriever
from haystack.components.builders import PromptBuilder
from haystack.components.generators import OpenAIGenerator

store = InMemoryDocumentStore()
store.write_documents([Document(content="Chapter 2 covers neural networks..."),
                       Document(content="Chapter 3 covers transformers...")])

template = """Answer using only these documents:
{% for d in documents %}- {{ d.content }}
{% endfor %}
Question: {{ question }}"""

pipe = Pipeline()
pipe.add_component("retriever", InMemoryBM25Retriever(document_store=store, top_k=3))
pipe.add_component("prompt",    PromptBuilder(template=template))
pipe.add_component("llm",       OpenAIGenerator(model="gpt-4o-mini"))
pipe.connect("retriever.documents", "prompt.documents")
pipe.connect("prompt.prompt",       "llm.prompt")

q = "What is chapter 2 about?"
out = pipe.run({"retriever": {"query": q}, "prompt": {"question": q}})
print(out["llm"]["replies"][0])
```

Serving it

Save the pipeline with `pipe.dumps()` to YAML, then expose it as a REST API with **Hayhooks** or deploy it on the managed deepset AI Platform.

Strengths

- Clear, explicit, debuggable pipelines
- Stable API, built for production
- Strong classic search (BM25, Elasticsearch)
- YAML serialisation and easy deployment

Watch out for

- Smaller community and fewer tutorials
- More verbose wiring for simple apps
- Fewer integrations than LangChain

Head-to-head comparison

Same problems, different philosophies. Ratings are relative to each other, not absolute.

Aspect	LangChain	LlamaIndex	Haystack
Primary focus	General LLM apps, agents, tool use	Ingesting, indexing and querying data (RAG)	Production search, QA and RAG pipelines
Core abstraction	Runnables + LCEL; agents as LangGraph graphs	Index, Retriever, Query Engine, Workflows	Components connected in a Pipeline graph
RAG out of the box	Good; you assemble pieces	Excellent; few lines of code	Very good; explicit wiring
Agents	Excellent (LangGraph)	Good (agents, Workflows)	Good (Agent component)
Integrations	Largest ecosystem	Large (LlamaHub)	Moderate, well-maintained
Learning curve	Moderate to steep	Easy for RAG	Moderate, very structured
API stability	Improved since 1.0, history of churn	Moderate	High since 2.0
Observability	LangSmith (first-party)	Callbacks, third-party tools	Tracing integrations, deepset platform
Deployment	LangGraph Platform, any Python server	LlamaCloud, any Python server	Hayhooks, YAML, deepset platform
Languages	Python, JavaScript/TypeScript	Python, TypeScript	Python
Best for	Chatbots with tools, multi-agent systems	Chat-with-docs, knowledge bases	Enterprise search and document QA

At a glance (relative scores out of 5)

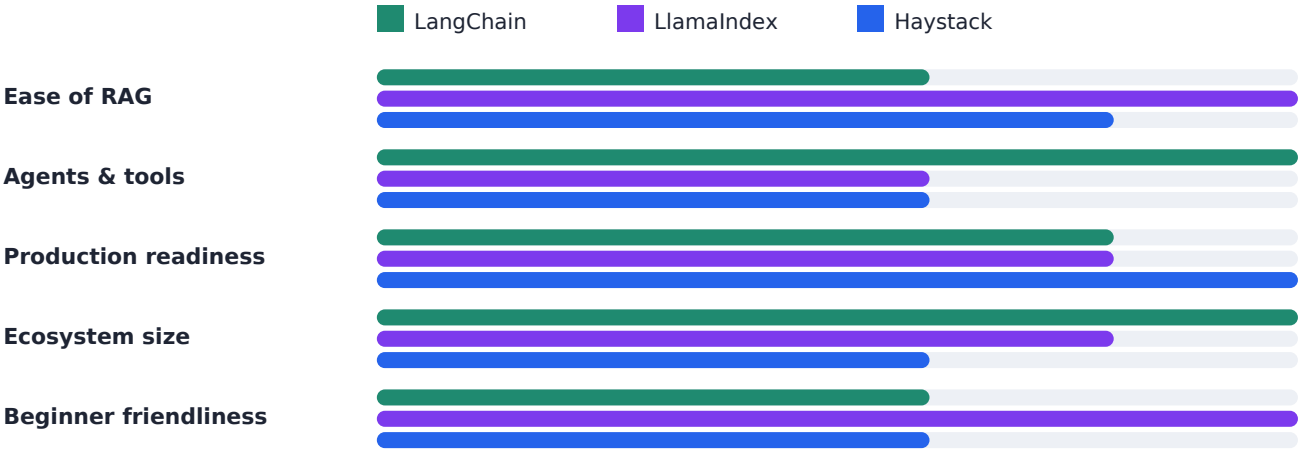


Figure 6. Indicative scores to guide choice; your project's needs matter more than the averages.

Official documentation

LangChain: python.langchain.com • LlamaIndex: docs.llamaindex.ai • Haystack: docs.haystack.deepset.ai

Which one should you choose?

Start from what you are building, not from the framework.

I want to chat with my PDFs / build a knowledge base — **LlamaIndex**

Fastest route from a folder of files to accurate answers with citations. Advanced chunking, re-ranking and LlamaParse help with messy real-world documents.

I'm building an agent that uses tools, APIs or multiple steps — **LangChain + LangGraph**

Best support for tool calling, memory, human-in-the-loop approval and multi-agent systems, with LangSmith to trace every step.

I need reliable search / QA in production at a company — **Haystack**

Explicit, testable pipelines, strong Elasticsearch/OpenSearch support, a stable API and YAML-based deployment.

I'm learning for jobs and interviews — **LangChain first**

It appears most often in job descriptions. Learn LlamaIndex next; the RAG concepts transfer directly to all three.

Combining them

These frameworks are not mutually exclusive. A common production pattern uses a LangGraph agent as the "brain" and plugs in LlamaIndex or Haystack pipelines as specialised retrieval tools.

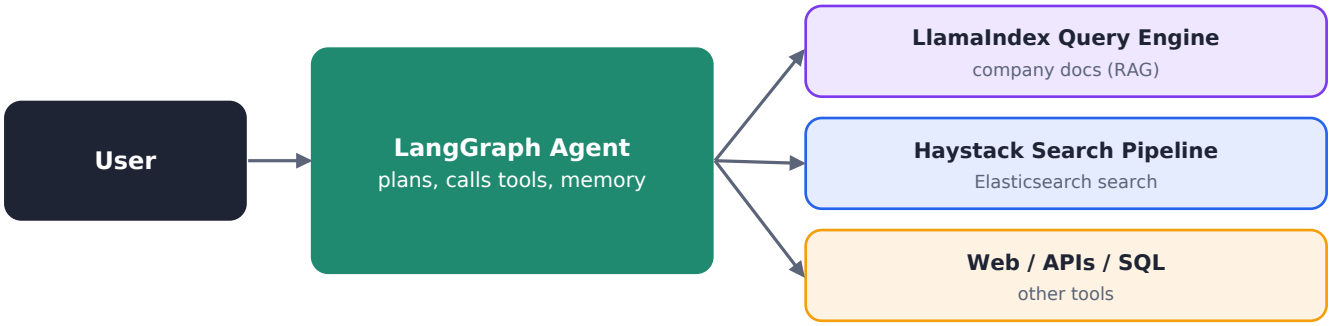


Figure 7. A hybrid architecture: LangGraph orchestrates; LlamaIndex and Haystack handle retrieval.

Quick tips

- **Learn the concepts once.** Loaders, chunking, embeddings, retrieval and prompting are the same everywhere.
- **Chunking matters most.** Bad splits cause more wrong answers than the choice of framework or model.
- **Evaluate early.** Keep 20 to 50 real questions with expected answers and test every change against them.
- **Check the version.** All three evolve quickly; prefer official docs over old blog posts.
- **Start simple.** A plain RAG pipeline often beats a complex agent. Add agents only when you need decisions or tools.